

Warden Flow — Node Documentation

Reverie Hacks 2026 · ML Prompt Engineering track

Warden Flow is **two** multi-node LLM workflows sharing one governance core. **DELIVER** turns a repository into reviewed delivery artifacts — a Dockerfile, a CI/CD pipeline, deploy manifests. **OPERATE** turns a raw production alert into a reviewed, minimal code fix. Neither does this with one large prompt. It does it as a chain of narrowly-scoped nodes, where each node runs one specialized LLM query, is forced to return a **structured object** (not free text), and hands that structure to the next node. The design thesis of the whole submission is:

A production fix is only as trustworthy as the reasoning behind it. You get trustworthy reasoning by decomposing the problem into small, individually-verifiable LLM queries with typed outputs – not by asking one model to do everything at once.

This document explains every node: what it does, the exact prompt that drives it, the model and budget it runs under, the structured output it must return, and — most importantly — **the specific failure mode each design choice is defending against**.

Where each track requirement is answered

Requirement	Where it lives
Where human input is necessary	Node A6 / OPERATE Node 4 — both are <code>HUMAN</code> steps with <code>approval: required</code> ; marked in amber on the flowchart PNGs. The workflow <i>halts</i> at both.
What queries are used with LLMs	The "The prompt" block under every node below — quoted from <code>warden/agents/definitions.py</code> , the file the code actually loads.
Which LLM model is used	The spec table at the top of every node. DELIVER runs <code>azure/gpt-5.6-sol</code> ; OPERATE runs <code>gemini-3.5-flash</code> . Both are declared in <code>manifests/</code> and swappable without a code change.
What each action does	The spec table (tools, scope, budget) plus the Output schema block — the typed object each node returns and hands to the next.
Reasoning behind each node	The "Prompt-engineering choices and the failure they defend against" bullets under every node.

Companion files: `warden-flow-ml-workflow.png` (the full flowchart — both workflows, every prompt, the revert loop), `warden-flow-deliver.png` and `warden-operate.png` (one-screen versions), `workflow-vs-single-prompt.md` (the same task, structured vs single prompt).

Part A — DELIVER (the proactive workflow)

Five nodes turn a repository into reviewed delivery artifacts. Where OPERATE reasons about a *running* system, DELIVER reasons about *source code* — but the spine is identical: one job per node, a typed output, scoped tools, a budget, and a human gate on the only step that writes anything.

Model. All five run on `azure/gpt-5.6-sol` (Azure OpenAI GPT-5.6, routed through LiteLLM) via the `DELIVER_MODEL` environment variable. Each manifest declares `gemini-3.5-flash` as its default, so the same workflow runs on either provider without touching orchestration code. The model is configuration; the structure is the design.

#	Node	LLM query	Structured output	Action taken
A1	Assess	What is this service, actually?	<code>RepoProfile</code>	Read the repo; generate nothing
A2	Containerize	What is the safe production image for it?	<code>Dockerfile</code>	Propose a Dockerfile
A3	Pipeline	How does it get built, checked and shipped?	<code>Pipeline</code>	Propose a CI/CD workflow
A4	Deploy plan	How does it run, and how is a bad rollout reversed?	<code>DeployPlan</code>	Propose manifests + rollback
A5	Verify artifacts	Is any of this unsafe to hand to a human?	<code>VerifyReport</code>	Gate: pass or list issues
A6	Human review	(no LLM) Is this safe to merge?	merge / close	A person decides; CI then deploys

Node A1 — Assess

Model	<code>azure/gpt-5.6-sol</code> (manifest default <code>gemini-3.5-flash</code>)
Budget	60,000 tokens · 15 tool calls — generous on calls, because reading the wrong three files is how every later node goes wrong
Tools	<code>list_source_files</code> , <code>read_source_file</code> (read-only)
Scope	<code>source:local:read</code> · blast radius 0 (<code>maxChangedLines: 0</code>)
Output schema	<code>RepoProfile</code>

Output schema

```
class RepoProfile(BaseModel):
    language: str
    framework: str          # "none" for a plain script – not invented
    entrypoint: str         # how the process actually starts
    ports: list[int]        # read from the code, not assumed
    build_system: str
    dependencies: list[str]
    notes: str              # what could NOT be determined from the files
```

The prompt

You are assessing a code repository so it can be containerized and deployed. You read; you generate nothing yet. Call `list_repo_files` first to see the layout, then `read_repo_file` on the files that actually tell you how the service runs – the dependency manifest, the entrypoint, and any existing Dockerfile or start script. Do not guess paths. Report only what the evidence supports: language and framework (say "none" for a plain script, do not invent one), the exact entrypoint, the port(s) it listens on, read from the code, not assumed. **If a fact is not in the files you read, say so in notes rather than filling it in. A containerizer that trusts a guessed port or entrypoint will build an image that cannot start.**

Prompt-engineering choices and the failure they defend against

- **"You read; you generate nothing yet."** The strongest temptation for a capable model handed a repository is to skip ahead and write the Dockerfile immediately. *Defends against:* a profile written backwards from an imagined Dockerfile rather than from the code.
- **notes as a first-class field for the unknown.** There is a designated place to put "I could not determine the port," which means the model does not have to choose between leaving a required field blank and inventing a value. *Defends against:* silent fabrication in `ports` or `entrypoint`, which produces an image that builds and then exits immediately.
- **"Say 'none' for a plain script, do not invent one."** Naming the specific fabrication we saw. Models reach for a framework because most repos have one. *Defends against:* a stdlib HTTP script being containerized as though it were Flask.
- **The consequence is stated, not just the rule.** "A containerizer that trusts a guessed port will build an image that cannot start" tells the model *why* precision matters here, which in practice moves behaviour more than the instruction alone.
- **Read-only scope in the manifest, blast radius 0.** The constraint is enforced by the tool proxy, not only requested in the prompt. *Defends against:* a prompt-level rule being the only thing standing between the node and a write.

Real-world note. On `github.com/mazzyy/testing-python`, Assess correctly reported a **Node/Vite frontend** despite the repository name saying "python". It read `package.json` rather than trusting the label — which is exactly the behaviour the `notes`-and-evidence framing is there to produce.

Node A2 — Containerize

Model	<code>azure/gpt-5.6-sol</code>
Budget	80,000 tokens · 10 tool calls
Tools	<code>read_source_file</code> (read-only)
Scope	<code>source:local:read</code> · 1 file, max 200 lines
Approval	<code>required</code> — this artifact reaches a human before it reaches a registry
Output schema	<code>Dockerfile</code>

Output schema

```
class Dockerfile(BaseModel):
    content: str # the complete file, ready to build
    base_image: str
    multistage: bool
    runs_as_nonroot: bool
    security_notes: list[str] # one entry per enforced control
```

The prompt

You write a PRODUCTION Dockerfile for the service described to you. Correctness and security are the whole job – a Dockerfile that builds but runs as root with an unpinned base is a failure, not a fix. Enforce every one of these, and record each in `security_notes`: **PIN the base image to a MINOR series** (e.g. `alpine:3.22`) and never `:latest`. **Do NOT pin a frozen patch tag** like `alpine:3.20.3` – that freezes the image at the CVE set of the day that tag was cut. **The series you pin MUST still be supported by its distribution.** Use a MULTI-STAGE build. Create and switch to a NON-ROOT user for the final stage. COPY only what runs; never bake a secret into a layer. Base the Dockerfile on the RepoProfile you were given – the real entrypoint, port and dependencies – not on a generic template.

Prompt-engineering choices and the failure they defend against

- `security_notes: list[str]` as a forced receipt. The model must enumerate the controls it applied. Asking for a list of what you did is a much stronger constraint than asking you to do it. *Defends against*: a plausible Dockerfile that quietly skips the non-root user.
- `multistage` and `runs_as_nonroot` as typed booleans. Two of the five controls are machine-checkable straight from the schema, and the benchmark scores them from the Dockerfile text rather than trusting the flag. *Defends against*: the model claiming a property it did not implement.
- "Base it on the RepoProfile, not a generic template." *Defends against*: the single most common LLM Dockerfile failure — emitting a well-known template for the wrong language, ignoring the profile entirely.
- The pin instruction is three clauses, not one. This is a scar. The original prompt said only "pin the base image to a specific minor version." The model complied — and pinned `alpine:3.20.3`, a frozen patch tag from an Alpine series that reached end of life in April 2026. The image was pinned exactly as instructed and carried **21 HIGH/CRITICAL vulnerabilities** with no upstream fixes available. *Defends against*: literal compliance with an underspecified rule. The prompt now distinguishes *pinning* from *pinning to something maintained*, because those are different properties and only one of them was ever asked for.

Node A3 — Pipeline

Model	azure/gpt-5.6-sol
Budget	80,000 tokens · 4 tool calls
Tools	none — this node generates; it reads nothing and runs nothing
Scope	none · 1 file, max 200 lines
Approval	required
Output schema	Pipeline

The prompt

You write a CI/CD pipeline that builds, checks and ships the container you were told about. The stages, in order: **build** the image; **test** the project's tests and fail the pipeline if they fail; **scan** the built image for known vulnerabilities and fail on high/critical findings – a pipeline that pushes an unscanned image is the hole this stage exists to close; **push** to the registry only on the repository's DEFAULT branch, never on a PR; **deploy** only after push, to the EXACT image this run built. Hard rules: secrets are referenced as `{{ secrets.NAME }}`, never hardcoded and never echoed. Grant an explicit minimal `permissions:` block. **NEVER hardcode a branch name in a gate.** You do not know this repository's default branch, and guessing `main` on a repo whose default is `master` produces a deploy job that skips forever – a failure that looks exactly like success. Gate on `github.event_name == 'push' && github.ref_name == github.event.repository.default_branch`. **Pin third-party actions to a tag that ACTUALLY EXISTS.** If you are not certain of a tag, do not invent one – run the tool from its official container image instead. **Every one of these is a thing that cannot be caught by reading the YAML – only by running it.**

Prompt-engineering choices and the failure they defend against

This node's prompt is almost entirely written from production failures, and each clause has a date attached.

- **The default-branch expression is given literally, not described.** The first live run generated `if: github.ref == 'refs/heads/main'` against a repository whose default branch is `master`. Verify passed it. A human merged it. The push and deploy jobs skipped silently and **nothing deployed** – and a skipped job renders indistinguishable from a successful one. *Defends against:* the highest-severity failure we found, precisely because it is invisible.
- **The failure mode is named, not just the rule.** "A deploy job that skips forever – a failure that looks exactly like success" is in the prompt because stating the rule alone had already failed once.
- **"Pin to a tag that ACTUALLY EXISTS... if unsure, use the container image."** The generated pipeline referenced `aquasecurity/trivy-action@0.33.1`; the real tag is `v0.33.1`, so GitHub could not resolve it and the scan job died in about two seconds. Offering an *alternative* ("use the container image") rather than only a prohibition gives the model a safe path when it is uncertain. *Defends against:* invented version tags, and the follow-on discovery that even the correct action rate-limits on shared runners.
- **The closing line — "only by running it."** An explicit statement that this node's output cannot be validated downstream. *Defends against:* the model relying on a later gate that structurally cannot catch these classes of error.
- **Zero tools.** This node cannot read the repository. It reasons only over the typed `RepoProfile` and `Dockerfile` handed to it. *Defends against:* the pipeline node re-deriving facts the earlier nodes already established, and disagreeing with them.

Result. After these guardrails were added, a fresh live run produced a pipeline gated on `github.event.repository.default_branch`, running Trivy from `aquasec/trivy:0.58.2` as a container, with `--ignore-unfixed` added on the model's own initiative. **No human edited that file.**

Node A4 — Deploy plan

Model	azure/gpt-5.6-sol
Budget	80,000 tokens · 4 tool calls
Tools	none — generates only
Scope	none · 3 files, max 300 lines
Approval	required
Output schema	DeployPlan

Output schema

```
class DeployPlan(BaseModel):
    manifests: list[Manifest] # path -> YAML
    strategy: str             # e.g. RollingUpdate maxSurge=1 maxUnavailable=0
    rollback: str             # one line: exactly how a bad rollout is reversed
```

The prompt

You write the Kubernetes manifests and the rollout plan to run this container in production. Enforce: **resource requests AND limits on every container** – an unbounded pod is how one workload takes down a node; **liveness and readiness probes** wired to the port the service actually uses; a `securityContext` that runs as non-root, drops all capabilities, and sets a read-only root filesystem where possible. For the container image, use a CLEARLY-LABELLED placeholder – for example `image: REPLACED_BY_CI`. Never invent an all-zeros digest, a fake-looking registry path, or `:latest`. **You are TOLD the deploy target; the comment on the placeholder must match how that target ships.** If the manifests are the ACTIVE deploy artifact, say the pipeline substitutes the pushed image into THIS file. If they are REFERENCE-ONLY (Container Apps / App Service run the container directly), say so plainly – do NOT claim CI substitutes this file when it does not. Choose a rollout strategy and state, in one line, exactly how a bad rollout is reversed.

Prompt-engineering choices and the failure they defend against

- **rollback: str** is a required field. Making the reversal path a schema field rather than a suggestion means no plan can be produced without one. *Defends against:* a rollout plan that has no answer to "and if this is wrong?"
- **A labelled placeholder is mandated, and the fakes are named.** "Never invent an all-zeros digest or a fake-looking registry path" *defends against:* the observed behaviour of filling an image field with something that looks real enough to pass review and cannot possibly pull.
- **Target-awareness is injected as data, not asked for.** The orchestrator computes the deploy target from configuration and injects a `deploy_summary()` paragraph into this prompt. The model is *told* whether its manifests will be applied. *Defends against:* the manifests claiming "CI substitutes this image" on an Azure Container Apps target where nothing ever applies them — a comment that would mislead every future reader.
- **"An unbounded pod is how one workload takes down a node."** The reason, in the prompt, next to the rule. Consistently more effective in our runs than the bare instruction.

Node A5 — Verify artifacts

Model	<code>azure/gpt-5.6-sol</code>
Budget	40,000 tokens · 0 tool calls
Tools	none — it reads the typed handoff and nothing else
Scope	none · blast radius 0
Output schema	<code>VerifyReport</code>

Output schema

```
class VerifyReport(BaseModel):
    passed: bool
    checks: list[str] # what was verified and found correct
    issues: list[str] # specific enough to fix without re-reading everything
```

The prompt

You are the last gate before a human review. You are given the generated Dockerfile, pipeline and deployment manifests. Validate them; you build and deploy nothing. Check the things that actually break in production: Dockerfile – pinned base, multi-stage, non-root, no secret baked into a layer. Pipeline – has a scan stage, references secrets rather than hardcoding them, least-privilege permissions, and **gates push/deploy on the repository's DEFAULT branch rather than a hardcoded name. Treat a literal branch gate as an ISSUE, not a pass.** Manifests – resource limits, probes, non-root securityContext, no `:latest`. Set `passed=false` if ANY critical check fails, and list every problem in `issues` – specific enough that a human can fix it without rereading everything. **Do not soften a real failure into a passing note; the whole point of this node is to catch what the generators missed.**

Prompt-engineering choices and the failure they defend against

- **Zero tool calls, by manifest.** This node physically cannot go and look at anything. It judges only the artifacts in front of it. *Defends against:* a verifier that wanders off, re-reads the repo, and forms a different opinion than the nodes it is checking.
- **"Do not soften a real failure into a passing note."** Written because a gate whose job is to approve has a gravitational pull toward approving. *Defends against:* a `passed: true` accompanied by an issues list that quietly contains a blocker.
- **`passed` is a boolean the orchestrator branches on,** not prose a human must interpret. *Defends against:* an ambiguous verdict being read optimistically by whoever is in a hurry.
- **Issues must be individually actionable.** *Defends against:* "some security concerns were noted," which costs a reviewer more time than no report at all.

The honest limitation — and why it is in the architecture on purpose. This node performs **static** checks. It reads text. It cannot know whether `aquasecurity/trivy-action@0.33.1` resolves on GitHub, whether a branch gate matches the repository's real default, or whether Alpine 3.20 went end-of-life in April. All three of those got past it, and all three were caught the moment the pipeline actually ran.

We treat that as a designed boundary rather than a bug. Verify is the cheap gate that runs in seconds on every generation; the **scan and deploy stages of the generated pipeline** are the expensive gate that runs on merge. The next version of this node closes the gap by reading the GitHub Actions result for the pull request it just opened and repairing the artifacts until the run is green — turning plausible YAML into proven-green YAML.

Node A6 — Human review (no LLM)

Model	none — no model call happens at this step
Tools	<code>open_pull_request()</code> was called by the workflow; the human uses GitHub
Approval	<code>blocking</code> — the workflow does not proceed until a person acts
Output	<code>merge</code> · or · <code>close</code>

What happens

The five generated artifacts are written to a new branch and opened as a pull request. That is the fleet's **only write primitive**. No node in DELIVER holds a registry credential, a cluster credential, or the ability to merge. It cannot push to a default branch. It cannot deploy. The single thing it can do is ask.

Why a human is here, and not somewhere else

- **This is the last point where a mistake is still cheap.** Everything before it is text in a branch. Everything after it is a running container. Putting the gate exactly at that boundary is the entire safety argument, and it is why `--apply` (build → push →

deploy without a PR) is a separate, explicit, deliberately inconvenient flag rather than a default.

- **The pull request is the review format engineers already use.** No new dashboard to learn, no bespoke approval queue. The diff, the CI checks and the discussion all live where they normally live.
- **It is what makes the generators safe to keep improving.** Because a person reads every artifact before it lands, we can iterate aggressively on the prompts without any single bad generation reaching production.
- **The verify node's report is written for this reader.** `issues` must be specific enough to act on without re-reading everything, because the person reading it has thirty seconds, not thirty minutes.

What the human sees. The Dockerfile, `.github/workflows/deliver.yml`, the `k8s/` manifests, and the `VerifyReport` summary. On merge, the pipeline the agent wrote runs — build, test, scan, push to the registry, deploy the exact commit image.

Part B — OPERATE (the reactive workflow)

The workflow at a glance

#	Node	LLM query	Structured output	Action taken
1	Triage	Is this real, is it a duplicate, is it worth escalating?	TriageVerdict	Gate: escalate or close
2	Diagnosis	What is the one root cause, and what evidence proves it?	Diagnosis	Produce hypothesis + evidence chain
3	Remediation	What is the smallest change that fixes exactly that root cause?	ProposedPatch	Open a pull request
4	Human Review	(no LLM) Is this fix safe to merge?	merge / reject signal	Human merges or closes the PR
5	Verification	Did the merged fix actually restore health?	free reasoning → tool call	Close incident, or open a revert PR

Three prompt-engineering principles run through all of it:

- One job per node.** Each prompt is told to decide *one* thing and explicitly told what *not* to do. A node that tries to diagnose *and* fix will do both worse.
- Typed output, enforced.** Nodes 1–3 are pinned to a Pydantic schema via the model runtime's `output_schema`. The model cannot return prose where a decision is required; it must fill fields.
- Structured handoffs.** A node does not re-read the raw world. It consumes the *typed output* of the node before it. The diagnosis's evidence chain is piped verbatim into the remediation prompt, so the fixer reasons over vetted facts, not raw logs.

Cross-cutting technique: structured output + structured handoff

This is the mechanism that makes the whole thing more than a chatbot with tools.

Structured output. Each decision node declares a schema:

```
class Diagnosis(BaseModel):
    hypothesis: str          # what is wrong, in one sentence
    root_cause: str          # the specific change or condition responsible
    evidence: list[Evidence] # facts supporting the hypothesis
    suggested_fix: str
    confidence: float        # 0.0–1.0
```

The runtime enforces this shape on the node's final answer. That does two things at once: it *forces the model to commit* to discrete claims (a `root_cause` field cannot be filled with "it's probably something in the config, hard to say"), and it makes the output *machine-checkable* by the next node and by the audit trail.

Structured handoff. The orchestrator does not paste the diagnosis text into the next prompt. It reassembles the *fields* into the remediation query:

```
prompt = (
    f"Hypothesis: {diagnosis.hypothesis}\n"
    f"Root cause: {diagnosis.root_cause}\n"
    f"Suggested fix: {diagnosis.suggested_fix}\n"
    f"Evidence:\n"
    + "\n".join(f"  - [{e.source}] {e.detail}" for e in diagnosis.evidence)
    + "\n\nOpen a pull request that fixes this."
)
```

The remediator therefore starts from a clean, cited root cause — not from 400 lines of crashloop logs it would have to re-interpret (and might re-interpret *differently* than the diagnostician did). Every handoff is a checkpoint where reasoning is frozen into typed facts.

Node 1 — Triage

Model	gemini-3.5-flash
Budget	20,000 tokens · 6 tool calls — the tightest tier in the fleet, because this node runs on <i>every</i> alert
Tools	get_alert_context, recall_similar_incidents (read-only)
Scope	memory:incidents:read · blast radius 0 (cannot write anything)
Output schema	TriageVerdict

Output schema

```
class TriageVerdict(BaseModel):
    severity: Severity          # critical | high | medium | low | noise
    escalate: bool              # True if the fleet should investigate
    duplicate_of: str | None    # existing incident id if this is a repeat
    reasoning: str              # one or two sentences on why
```

The prompt

You are the first responder for a production estate. A signal has arrived. Decide three things and nothing else: how severe this is; whether it is a duplicate of an incident already open; whether it is worth waking the rest of the fleet for. Call `get_alert_context` first. If the signal has a recognisable failure signature, call `recall_similar_incidents` before deciding. [...] Understand what escalation costs HERE, because it is not what it costs on a human rota. Escalating does not wake a person at 3am. It wakes three agents, for a few cents and about thirty seconds [...]. So the asymmetry runs the other way from the one you are used to: escalating something harmless is cheap, and failing to escalate something real means nobody looks at it at all. In particular, "no user impact right now" is not the same as "no incident". A blocked or failed rollout is a latent outage even while every replica is serving [...].

Prompt-engineering choices and the failure they defend against

- The `escalate` boolean is a **hard routing gate**. Making the model commit to a typed yes/no — not a paragraph — lets the orchestrator cheaply terminate the workflow at Node 1 for the 90% of alerts that are noise. *Defends against*: burning the expensive diagnosis budget on non-incidents.

- **Explicitly re-anchoring the cost model** ("escalating wakes agents, not a person"). An LLM's prior, learned from human ops writing, is that paging is expensive and should be rare. That prior is *wrong* here and produces over-suppression. *Defends against*: a false negative where a real fault is silently closed because the model was being "considerate."
- **The named counter-example** ("no user impact now ≠ no incident; a blocked rollout is a latent outage"). *Defends against*: the single most likely triage miss in this estate — a stuck rollout that looks healthy because the old revision is still serving.
- **"Decide three things and nothing else."** *Defends against*: scope creep, where triage starts diagnosing and spends its tiny budget doing the next node's job badly.

Node 2 — Diagnosis

Model	gemini-3.5-flash
Budget	120,000 tokens · 25 tool calls — the largest in the fleet; this is where reasoning has to earn its keep
Tools	describe_workload, get_workload_logs, recent_deploys, query_metrics — all read-only
Scope	cluster:demo:read · blast radius 0
Output schema	Diagnosis

The prompt

You are diagnosing a production failure. You can read; you cannot change anything, anywhere. Do not propose a fix as though you could apply it. Work from evidence, in roughly this order: `describe_workload` — what state is it actually in? `get_workload_logs` — what did it say before it died? `recent_deploys` — what changed just before this started? `query_metrics` — confirm the blast radius. **Your output is a hypothesis with an explicit evidence chain. Every claim in `root_cause` must be traceable to something a tool actually returned; cite the specific log line or field, not a paraphrase.** If the evidence does not support a confident conclusion, say so in the confidence score rather than inventing a tidy story. [...] Diagnose ONE root cause — the one that explains the failure in the logs. Do not list every unusual thing you noticed. [...] an unfamiliar image, an inline command or a hand-rolled entrypoint may be entirely deliberate. Saying "this also looks wrong to me" invites a fix that breaks something which was working.

Prompt-engineering choices and the failure they defend against

- **Forced evidence chain — the single most important technique in the workflow.** The `evidence: list[Evidence]` field, plus "every claim in `root_cause` must be traceable to a tool output; cite the specific line, not a paraphrase," turns the model from a plausible-story generator into an investigator that must show its work. *Defends against*: hallucinated root causes — the failure mode that makes a single naive prompt unsafe to act on.
- **The prescribed tool order** (state → logs → change → metrics). *Defends against*: jumping to a conclusion from the first log line before establishing what actually changed.
- **`confidence: float` + "say inconclusive rather than invent a tidy story."** Calibrated uncertainty is a first-class output, not an afterthought. *Defends against*: false confidence propagating downstream into a wrong fix that a reviewer trusts.
- **"Diagnose ONE root cause; an unfamiliar image may be deliberate."** This is a scar from a real run: given a workload with a hand-rolled entrypoint, the model flagged the image as "wrong" and proposed replacing it, which would have broken a working cluster. *Defends against*: over-diagnosis, where noticing many odd things leads to a destructive multi-part fix.
- **"You can read; you cannot change anything."** Reinforced by a zero-write scope in the manifest. *Defends against*: the model proposing an apply-style action the node has no authority to take.

Node 3 — Remediation

Model	gemini-3.5-flash
Budget	120,000 tokens · 20 tool calls
Tools	list_repo_files, read_repo_file, propose_patch
Scope	repo:estate-gitops:read, repo:estate-gitops:write-pr — no cluster write, cannot merge
Blast radius	max 3 files · max 12 changed lines, enforced at the write path
Approval	required — this is the hard gate into Node 4
Output schema	ProposedPatch

Input (structured handoff from Node 2): hypothesis + root cause + suggested fix + the full evidence list, reassembled by the orchestrator.

The prompt

You turn a diagnosis into the smallest change that fixes it. You cannot reach the cluster. Your only action is `propose_patch`, which opens a pull request that a human reviews and merges. Write for that human. Call `list_repo_files` FIRST to find the exact path, then `read_repo_file`. Do not guess paths [...]. Return the complete new contents of the file you read — not a diff, not a fragment. Base it on what `read_repo_file` actually returned: never reconstruct a file from memory, because you will silently drop the parts you did not think to include. Change as little as possible [...]. Patch ONLY what the diagnosis names as the root cause. Never change an image tag, delete a command block, or restructure a manifest as a side effect [...]. The rationale you pass becomes the pull request body. It should let a reviewer who has not seen the incident decide in thirty seconds whether to merge [...]. If you are not confident the patch is right, say that in the rationale.

Prompt-engineering choices and the failure they defend against

- "`list_repo_files` FIRST, then `read_repo_file`, do not guess paths." *Defends against:* hallucinated file paths — the model confidently editing `k8s/deployment.yaml` when the file is actually `manifests/checkout/deploy.yaml`.
- "Return the complete new file contents; never reconstruct from memory." *Defends against:* the model regenerating a file from its training prior and silently dropping the fields it didn't think to include — a subtle, dangerous corruption.
- "Patch ONLY the root cause; never change an image tag or entrypoint as a side effect," backed by the 12-line blast-radius cap enforced in code. *Defends against:* collateral edits that fix the reported bug while breaking something that was working — the model cannot verify a different image exists, so it must not touch it.
- "The rationale becomes the PR body; write so a reviewer decides in thirty seconds; state what you couldn't verify." The prompt is explicitly optimizing the output *for the human in Node 4*. *Defends against:* an opaque diff a reviewer can't safely approve — false confidence is unmergeable, an honest "here's what I couldn't verify" is safe behind review.

Node 4 — Human-in-the-Loop Review

This node runs no LLM. That is the point. It is the required human-input stage of the workflow, and it is a first-class node rather than a footnote because the system is *architecturally* blocked here.

How the block works. The remediator's manifest sets `approval: required`. After Node 3 opens the PR, the orchestrator sets the incident to `awaiting_merge` and **returns** — the workflow halts. Node 5 (Verification) is only ever triggered by the GitHub

webhook that fires *after a human merges*. No human action, no verification, no closure. The human is not advisory; they are load-bearing.

```
# orchestrator: after remediation
incident.status = IncidentStatus.awaiting_merge
# ... the initial alert path does NOT run the verifier.
# Only the post-merge webhook path sets verify=True.
```

The prompt engineering that makes this node work is all upstream. A human review node is only as good as the artifact it reviews. Everything Nodes 2 and 3 were told to do — cite specific evidence, produce a minimal diff, keep the blast radius tiny, write a 30-second PR body, confess low confidence — exists to make *this* decision fast and safe. The human is reviewing a decision-ready object, not raw model output.

Why it belongs in the workflow diagram. The human's merge-or-reject is the workflow's ground-truth signal: it is the point where machine reasoning is validated by human judgment before anything touches production. It is also the governance seam — the reason no agent in the fleet needs production write access. Its only write primitive is *proposing*; a human owns *applying*.

Node 5 — Verification

Model	gemini-3.5-flash
Budget	40,000 tokens · 15 tool calls
Tools	query_metrics, get_workload_status, request_revert
Scope	cluster:demo:read, repo:estate-gitops:write-pr (can propose a revert, cannot apply one)
Output	free reasoning ending in either "recovered" or a request_revert call
Runs	only after the human merge in Node 4

The prompt

```
The fix has merged and synced. Decide whether it worked. Check get_workload_status and query_metrics. Compare against what the incident described – a service that is healthy for a different reason has not been fixed. If it recovered, close the incident. If it did not, call request_revert with a clear reason. Do not wait and hope. Rolling back a change that did not help is cheap; leaving a broken service in production while you deliberate is not.
```

Prompt-engineering choices and the failure they defend against

- **"Compare against what the incident described — healthy for a different reason is not fixed."** *Defends against:* a false-resolve, where the service happens to look green (e.g. a restart, a traffic dip) and the node closes an incident the patch never actually fixed.
- **"Do not wait and hope ... reverting is cheap."** A deliberate action-bias, and the reason this node holds a `write-pr` scope for revert. *Defends against:* an ambiguous half-recovery being left to fester in production.
- **No output schema.** Unlike Nodes 1–3, the verifier's job ends in a *tool call* (`request_revert`) or a plain close, so it reasons freely and acts, rather than filling a form. The structure here lives in the tool contract, not a response schema.

Model and budget selection

Node	Model	Token budget	Tool-call cap	Why this tier
Triage	gemini-3.5-flash	20,000	6	Runs on every alert; must be the cheapest. Tight cap forces a fast gate.
Diagnosis	gemini-3.5-flash	120,000	25	The reasoning core. Needs room to read four tools and build an evidence chain.
Remediation	gemini-3.5-flash	120,000	20	Must read the real repo file whole and reason about a minimal edit.
Human Review	—	—	—	No model. Human judgment.
Verification	gemini-3.5-flash	40,000	15	Bounded check against metrics; cheaper than diagnosis, richer than triage.

The per-node **budget cap is the real cost lever** in this workflow: a high-volume node is capped tight, the two reasoning nodes get the large budget, verification sits in between. Because every node is declared in a version-controlled manifest, the *model* itself is a one-line change — the high-volume Triage classifier is the obvious candidate to route to a smaller, cheaper model (e.g. Gemma) without touching a line of orchestration code. Model choice and cost control are configuration, not code.

Why nodes, not one prompt

Every design choice above is a defense against a way a single "here's the error, fix it" prompt fails: it hallucinates a root cause it can't support, guesses a file path, reconstructs a file from memory and drops fields, makes collateral edits it can't verify, and reports the whole thing with unearned confidence. Warden Flow doesn't ask the model to be more careful — it *structures the task* so those failures have nowhere to hide: read-only diagnosis with a forced evidence chain, a fixer that starts from vetted facts and is capped to a 12-line diff, and a human gate on the only step that changes anything. The companion comparison document demonstrates each of these failures happening, and not happening, on the same incident.